Index

A

acceleration of rate of change in software development ecosystem, 268
accessibility, 59
accidental architecture anti-pattern, 133
accidental complexity, 354
accountability, 61
achievability, 60
ACID transactions, 132
  in service-based architecture, 177
  in services of service-based architecture, 168
actions provided by presention tools, 321
actor/actions approach to designing components, 111
  in Going, Going, Gone case study, 112
actual productivity (of development teams), 335
adaptability, 62
administrators (network), 129
ADR-tools, 285
ADRs (architecture decision records), 285-295
  as documentation, 293
  auction system example, 294
  basic structure of, 285
  compliance section, 290
  context section, 288
  decision section, 288
  draft ADR, request for comments on, 287
  notes section, 291
  status, 286
  storing, 291
  title, 286
  using for standards, 293
Agile development
  Agile Story risk analysis, 308
  creation of just-in-time artifacts, 317
  extreme programming and, 15
  software architecture and, 18, 101
agility
  process measures of, 81
  rating in service-based architecture, 176
  versus time to market, 67
Ambulance pattern, 312
analyzability, 62
animations provided by presentation tools, 321
anti-patterns
  Big Ball of Mud, 85, 120
  Bullet-Riddled Corpse in corporate presentations, 322
  Cookie-Cutter, 321
  Covering Your Assets, 282
  Email-Driven Architecture, 283
  Entity Trap, 110
  Frozen Caveman, 30
  Generic Architecture, 65
  Groundhog Day, 282
  Irrational Artifact Attachment, 316
  Ivory Tower Architect, 74, 351
anvils dropping effects, 321
Apache Camel, 186
Apache Ignite, 213
Apache Kafka, 146
Apache ODE, 186
Apache Zookeeper, 157
API layer
  in microservices architecture, 249
  in service-based architecture, 167

security risks of Diagnostics System API gateway, 313
application logic in processing units, 213
application servers
scaling, problems with, 211
vendors battling with database server vendors, 235
application services, 237
ArchiMate, 319
architects (see software architects)
architectural extensibility, 32
in broker topology of event-driven architecture, 182
architectural fitness functions, 17
architectural thinking, 23-36
analyzing trade-offs, 30
architecture versus design, 23-25
balancing architecture and hands-on coding, 34
self-assessment questions, 373
understanding business drivers, 34
architecturally significant, 284
architecture by implication anti-pattern, 133
architecture characteristics, 55-64
about, 55-58
analyzing for components, 109
cross-cutting, 59
defined, self-assessment questions, 374
definitions of terms from the ISO, 61
in distributed architecture Going, Going, Gone case study, 274
fitness functions testing
cyclic dependencies example, 84-86
distance from main sequence example, 86-88
governance of, 82
identifying, 65-75
design versus architecture and trade-offs, 74
extracting from domain concerns, 65-67
extracting from requirements, 67-69
self-assessment questions, 374
Silicon Sandwiches case study, 69-74
incorporating into Going, Going, Gone component design, 114
measuring, 77-82
operational measures, 78
process measures, 81
structural measures, 79-81
measuring and governing, self-assessment questions, 375
operational, 58
partial listing of, 58
ratings in event-driven architecture, 207-209
ratings in layered architecture, 139
ratings in microkernel architecture, 160
ratings in microservices architecture, 263-265
ratings in orchestration-driven service-oriented architecture, 241
ratings in pipeline architecture, 146
ratings in service-based architecture, 174
ratings in space-based architecture, 233
scope of, 91-98
architectural quanta and granularity, 92-98
coupling and connascence, 92
self-assessment questions, 375
structural, 59
in synchronous vs. asynchronous communications between services, 270
trade-offs and least worst architecture, 63
architecture decision records (see ADRs)
architecture decisions, 6, 281-295
anti-patterns, 281-284
Covering Your Assets, 282
Email-Driven Architecture, 283
Groundhog Day, 282
architecturally significant, 284
architecture decision records (ADRs), 285-295
self-assessment questions, 380
architecture fitness function, 83
architecture katas
origin of, 68
reference on, 372
Silicon Sandwiches case study, 69-74
architecture partitioning, 102
architecture quantum, 91
architectural quanta and granularity, 92-98
Going, Going, Gone case study, 95-98
architectural quanta in microservices, 265
architecture quanta in event-driven architecture, 208
architecture quanta in space-based architecture, 234

choosing between monolithic and distributed architectures in Going, Going, Gone component design, 115-116
in orchestration-driven service-oriented architecture, 242
quanta boundaries for distributed architecture Going, Going, Gone case study, 276
separate quanta in service-based architecture, 175
architecture risk, analyzing, 297-314
Agile story risk analysis, 308
risk assessments, 298
risk matrix for, 297
risk storming, 302-308
consensus, 304
identifying areas of risk, 303
mitigation of risk, 307
risk storming examples, 308-314
availability of nurse diagnostics system, 310
elasticity of nurse diagnostics system, 312
nurse diagnostics system, 308
security in nurse diagnostics system, 313
self-assessment questions, 380
Architecture Sinkhole anti-pattern, 138
architecture sinkhole anti-pattern
microkernel architecture and, 161
architecture styles, 119-132
choosing the appropriate style, 267-277
decision criteria, 269-271
distributed architecture in Going, Going, Gone case study, 274-277
monolithic architectures in Silicon Sandwiches case study, 271-274
self-assessment questions, 379
shifting fashion in architecture, 267-268
defined, 119
fundamental patterns, 119-123
monolithic versus distributed architectures, 123-132
self-assessment questions, 376
architecture vitality, 9
archivability, 59
ArchUnit (Java), 36, 87
fitness function to govern layers, 87
argumentativeness or getting personal, avoiding, 351
armchair architects, 328
arrows indicating direction of risk, 301
The Art of War (Sun Tzu), 350
asynchronous communication, 254, 270
in event-driven architecture, 196-197
in microservices implementation of Going, Going, Gone, 276
asynchronous connascence, 92, 94
auditability
in Going, Going, Gone case study, 96
performance and, 67
authentication/authorization, 59
authenticity, 61
auto acknowledge mode, 202
automation
leveraging, 35
on software projects, drive toward, 82
availability, 57
basic availability in BASE transactions, 132
in Going, Going, Gone case study, 97
implicit architecture characteristic, 73
in Going, Going, Gone: discovering comm-ponents case study, 114
in nurse diagnostics system risk storming example, 310
Italy-ility and, 60
in layered architecture, 141
negotiating with business stakeholders about, 349
nines of, 349
performance and, 67
in pipeline architecture, 148
rating in service-based architecture, 176
reliability versus, 60

B

Backends for Frontends (BFF) pattern, 273
bandwidth is infinite fallacy, 126
BASE transactions, 132
in service-based architecture, 177
basic availability, soft state, eventual consistency (see BASE transactions)
Big Ball of Mud anti-pattern, 85, 120
bottleneck trap, 34
bounded context, 93, 94
for services in microservices
data isolation with, 249
microservices and, 245
in microservices architecture, 247
granularity for services, 248

user interface as part of, 253
broadcast capabilities in event-driven architecture, 203
broker topology (event-driven architecture), 180-185
benefits and disadvantages of, 185
example, 182
Brooks' law, 335
Brooks, Fred, 68, 335, 372
Brown, Simon, 102, 318
bug fixes, working on, 35
build in animations, 321
build out animations, 321
Building Evolutionary Architectures (Ford et al.), 16, 82, 91
Bullet-Riddled Corpse anti-pattern, 322
business and technical justifications for architecture decisions, 282, 343
business delegate pattern, 136
business domains
knowledge of, 11
in layered architecture, 135
business drivers, understanding, 34
business layer
in layered architectures, 133
shared objects in, 136
Business Process Execution Language (BPEL), 187
business process management (BPM) engines, 187
business rules layer, 104
business stakeholders, negotiating with, 348

C

C's of architecture, 353
C4 diagramming standard, 318
caching
data collisions and caches in space-based architecture, 224
data pumps in caches in space-based architecture, 220
named caches and data readers, 222
named caches in space-based architecture, 216
near-cache considerations in space-based architecture, 230
replicated vs. distributed in space-based architecture, 227-230
capabilities, new, and shifting fashion in architecture, 268
capacity, 61
career path, developing, 365-372
developing a personal radar, 367-371
parting advice, 372
self-assessment questions, 381
twenty-minute rule, 365-367
using social media, 371
chaos engineering, 88
Chaos Gorilla, 88
Chaos Monkey, 88
The Checklist Manifesto (Gawande), 89, 338
checklists, leveraging, 338-343
developer code completion checklist, 340
software release checklist, 342
unit and functional testing checklist, 341
choreography
of bounded context services in microservices, 248
in microservices' communication, 256
circuit breakers, 124
clarity, 355
classes, representation in C4 diagrams, 319
classpath (Java), 40
client acknowledge mode, 202
client/server architectures, 121
browser and web server, 122
desktop and database server, 122
three-tier, 122
closed versus open layers, 135
cloud, space-based architecture implementations on, 226
code reviews by architects, 36
coding, balancing with architecture, 34
coexistence, 61
cohesion, 93
functional, 92
collaboration, 355
of architects with other teams, 74
color in diagrams, 320
Common Object Request Broker Architecture (CORBA), 122
communication, 355
communicating architecture decisions effectively, 283
communication between services, 270
in microservices architecture, 254-263

in microservices implementation of Going, Going, Gone, 276
communication connascence, 93
compatibility
defined, 61
interoperability versus, 60
compensating transaction framework, 262
competing consumers, 209
competitive advantage, translation to architecture characteristics, 67
complexity in architecture, 353
component-based thinking, 99-116
choosing between monolithic and distributed architectures in Going, Going, Gone component design, 115-116
component design, 110-112
component identification flow, 108-109
component scope, 99-100
developers' role, 108
granularity of components, 110
self-assessment questions, 375
software architect's role, 101-107
components
defined, 99, 101
representation in C4 diagrams, 318
concert ticketing system example (space-based architecture), 231
conciseness, 355
confidentiality, 61
configurability, 59
Conformity Monkey, 88
connascence
about, 92
asynchronous, 94
synchronous, in high functional cohesion, 93
connected components, 80
consistency, eventual, 132
constraints, communication by architect to development team, 325
construction techniques, architecurally significant decisions impacting, 285
Consul, 157
consumer filters, 144
containers in C4 diagramming, 318
context
architecture katas and, 68
bounded context and, 245
bounded context in domain-driven design, 94
context section of ADRs, 288
indicating in larger diagram using representational consistency, 315
representation in C4 diagrams, 318
continuity, 58
continuous delivery, 14
contracts
data pumps in space-based architecture, 220
maintenance and versioning, 132
in microkernel architecture, 158
in stamp coupling resolution, 126
control freak architects, 327
Conway's law, 103, 133
orchestration engine and, 238
Cookie-Cutter anti-pattern, 321
core system in microkernel architecture, 150-153
correlation ID, 204
cost
justification for architecture decisions, 283
in orchestration-driven service-oriented architecture, 243
overall cost in layered architectures, 140
overall cost in microkernel architecture, 160
overall cost in pipeline architecture, 147
overall cost in service-based architecture, 176
for risk mitigation, 307
in space-based architecture, 234
transport cost in distributed computing, 130
coupling
and connascence, 92
decoupling of services in microservices, 247
negative trade-off of reuse, 246
reuse and, in orchestration-driven service-oriented architecture, 241
Covering Your Assets anti-pattern, 282
Crap4J tool, 81
critical or important to success (architecture characteristics), 75
cross-cutting architecture characteristics, 59
cube or door transitions, 321
customizability, architecture characteristics and, 72
cyclic dependencies between components, 84-86
cyclomatic complexity

calculating, 79
good value for, 81
removal from core system of microkernel architecture, 150

D

data
deciding where it should live, 270
preventing data loss in event-driven architecture, 201-203
software architecture and, 19
data abstraction layer, 223
data access layer, 223
data collisions, 224-226
cache size and, 226
formula to calculate probable number of, 224
number of processing unit instances and, 226
data grid, 215
data isolation in microservices, 249
data meshes, 356
"Data Monolith to Data Mesh" article (Fowler), 356
data pumps, 213, 219
data reader with reverse data pump, 223
in domain-based data writers, 221
data readers, 213, 222
data writers, 213, 221
database entities, user interface frontend built on, 111
Database Output transformer filter, 146
database server, desktop and, 122
databases
ACID transactions in services of service-based architecture, 168
component-relational mapping of framework to, 111
data pump sending data to in space-based architecture, 220
licensing of database servers, problems with, 235
in microkernel architecture core system, 151
in microkernel architecture plug-ins, 156
in orchestration-driven service-oriented architecture, 242
partitioning in service-based architecture, 169-171
removing as synchronous constraint in space-based architecture, 212
scaling database server, problems with, 211
in service-based architecture, 164
transactions in service-based architecture, 177
variants in service-based architecture, 166
DDD (see domain-driven design)
demonstration defeats discussion, 350
dependencies
architecturally significant decisions impacting, 284
cyclic, modularity and, 84-86
timing, modules and, 41
deployability
low rating in layered architecture, 140
process measures of, 81
rating in microkernel architecture, 161
rating in orchestration-driven service-oriented architecture, 242
rating in pipeline architecture, 147
rating in service-based architecture, 176
deployment
automated deployment in microservices architecture, 263
deployment manager in space-based architecture, 219
physical topology variants in layered architecture, 134
design
architecture versus, 23
versus architecture and trade-offs, 74
understanding long-term implication of decisions on, 123
design principles in software architecture, 7
developer code completion checklist, 340
developer flow, 362
developers
drawn to complexity, 354
negotiating with, 351
role in components, 108
roles in layered architecture, 133
development process, separation from software architecture, 14, 101
development teams, making effective, 325-346
amount of control exerted by sotware architect, 331-335
leveraging checklists, 338-343
developer code completion checklist, 340

software release checklist, 342
unit and functional testing checklist, 341
self-assessment questions, 381
software architect personality types and, 326-331
software architect providing guidance, 343-346
team boundaries, 325
team warning signs, 335-338
DevOps, 3
adoption of extreme programming practices, 15
intersection with software architecture, 17
diagramming and presenting architecture, 315-324
diagramming, 316-321
guidelines for diagrams, 319
standards, UML, C4, and ArchiMate, 318
tools for, 316
presenting, 321-324
incremental builds of presentations, 322
infodecks vs. presentations, 324
invisibility, 324
manipulating time with presentation tools, 321
slides are half of the story, 324
representational consistency, 315
self-assessment questions, 380
diffusion of responsibility, 337
direction of risk, 300
directory structure for storing ADRs, 292
dissolve transitions and animations, 321
distance from the main sequence metric, 86-88
distributed architectures
domain partitioning and, 107
in Going, Going, Gone case study, 274-277
microkernel architecture with remote access plug-ins, 156
microservices, 247
monolithic architectures versus, 123-132, 270
fallacies of distributed computing, 124-131
in Going, Going, Gone case study, 115-116
other distributed computing considerations, 131
orchestration and choreography of services in, 177
in service-based architecture style, 175
stamp coupling in, 126
three-tier architecture and network-level protocols, 122
Distributed Component Object Model (DCOM), 122
distributed systems, 121
distributed transactions, 132
distributed vs. replicated caching in space-based architecture, 227-230
divide and conquer rule, 350
do and undo operations in transactions, 263
documentation, ADRs as, 293
domain partitioning (components)
defined, 104
in microkernel architecture, 161
in service-based architecture style, 174
in space-based architecture, 234
in Silicon Sandwiches monolithic architectures case study, 271
in Silicon Sandwiches partitioning case study, 107
technical partitioning versus, 249
domain-driven design (DDD), 94
component partitioning and, 104
event storming, 112
influence on microservices, 245
user interface as part of bounded context, 253
domain/architecture isomorphism, 256
domains
developers defining, 245
domain areas of applications, risk assessment on, 299
domain changes and architecture styles, 268
domain concerns, translating to architecture characteristics, 65-67
domain services in service-based architecture, 163, 168
domain-based data readers, 223
domain-based data writers, 221
domain-centered architecture in microservices, 265
inspiration for microservices service boundaries, 248
in technical and domain-partitioned architectures, 104

door or cube transitions, 321
driving characteristics, focus on, 65
duplication, favoring over reuse, 246
Duration Calculator transformer filter, 146
Duration filter, 146
dynamic connascence, 92
DZone Refcardz, 366

E

Eclipse IDE, 150, 158
effective architects, 330
effective teams (see development teams, making effective)
effects offered by presentation tools, 321
elastic scale, 13
elasticity, 70, 97
  being pragmatic, yet visionary about, 356
  in Going, Going, Gone case study, 96
  low rating in layered architecture, 141
  low rating in pipeline architecture, 148
  rating in microservices architecture, 264
  rating in orchestration-driven service-oriented architecture, 243
  rating in service-based architecture, 176
  rating in space-based architecture, 233
  risks in nurse diagnostics system example, 312
electronic devices recycling example (service-based architecture), 172-173
Email-Driven Architecture anti-pattern, 283
engineering practices, software architecture and, 14
Enterprise 2.0 (McAfee), 371
entity objects, shared library in service-based architecture, 170
entity trap, 110, 249
error handling in event-driven architecture, 197-200
errors (user), protection against, 61
essential complexity, 354
Evans, Eric, 94
event broker, 181
event mediators, 185
  delegating events to, 187
event processor, 181, 186
event queue, 186
event storming in component discovery, 112
event-driven architecture, 179-209
  architecture characteristics ratings, 207-209
  asynchronous capabilities, 196-197
  broadcast capabilities, 203
  choosing between request-based and event-based model, 206
  error handling, 197-200
  preventing data loss, 201-203
  request-reply messaging, 204
  self-assessment questions, 377
  topology, 180
    broker topology, 180-185
    mediator topology, 185-195
events
  commands versus in event-driven architecture, 195
  use for asynchronous communication in microservices, 255
eventual consistency, 132
eviction policy in front cache, 230
evolutionary architectures, 83
  event-driven architecture, 209
  microservices, 264
expectations of an architect, 8-13
explicit versus implicit architecture characteristics, 57
extensibility, 59
  rating in microkenel architecture, 161
extreme programming (XP), 14, 82

F

fallacies of distributed computing, 124-131
  bandwidth is infinite, 126
  latency is zero, 125
  the network is reliable, 124
  the network is secure, 127
  the network is homogeneous, 131
  the topology never changes, 128
  there is only one administrator, 129
  transport cost is zero, 130
fast-lane reader pattern, 136
fault tolerance
  layered architecture and, 141
  microkernel architecture and, 160
  pipeline architecture and, 148
  rating in event-driven architecture, 209
  rating in microservices arhitecture, 263
  rating in service-based architecture, 176
  reliability and, 61
feasibility, 72
federated event broker components, 181

filters
in pipeline architecture, 143
types of, 144
in pipeline architecture example, 146
First Law of Software Architecture, 19
fitness functions, 17, 36, 83-89
flame effects, 321
flow, state of, 362
Foote, Brian, 120
Ford, Neal, 30, 321, 354, 372
Fowler, Martin, 1, 245, 248, 356
front cache, 230
front controller pattern, 258
frontends
Backends for Frontends (BFF) pattern, 273
in microservices architecture, 253-254
Frozen Caveman anti-pattern, 30
full backing cache, 230
functional aspects of software, 62
functional cohesion, 92
high, 93
functions or methods, formula for calculating cyclomatic complexity, 79

G

Gawande, Atul, 89, 338
Generic Architecture (anti-pattern), 65
Going, Going, Gone case study, 95-98
using distributed architecture, 274-277
Going, Going, Gone: discovering components case study, 112-115
governance for architecture characteristics, 82
granularity
architectural quanta and, 92-98
for services in microservices, 248
Groundhog Day anti-pattern, 282
group potential, 335

H

Hawthorne effect, 340
Hazelcast, 213
creating internal replicated data grid with, 216
logging statements generated with, 217
heterogeneity
enforced, in microservices architecture, 255
heterogeneous interoperability in microservices, 255
Hickey, Rich, 32
high functional cohesion, 93

I

"-ilities", 4
implicit architecture characteristics
explicit characteristics versus, 57
modularity and, 38
in Silicon Sandwiches case study, 73
incremental builds for presentations, 322
infodecks versus presentations, 324
InfoQ website, 366
infrastructure services, 237
initial components, identifying, 108
initiating event, 181, 186
lack of control over workflow associated with, 185
installability, 59
portability and, 62
integrity, 61
IntelliJ IDEA IDE, 158
interfaces, architecturally significant decisions impacting, 285
International Organization for Standards (ISO)
definitions of software architecture terms, 61
functional aspects of software, 62
internationalization (i18n), architecture characteristics and, 73
interoperability, 61, 97
compatibility versus, 60
services calling each other in microservices, 255
interpersonal skills for architects, 12
Inverse Conway Maneuver, 104, 107
invisibility in presentations, 324
Irrational Artifact Attachment anti-pattern, 316
Isis framework, 111
Italy-ility, 60
Ivory Tower Architect anti-pattern, 74, 351

J

Janitor Monkey, 88
Java
Isis framework, 111
no name conflicts in Java 1.0, 39
three-tier architecture and, 123
JDepend tool, 86
Jenkins, 158
Jira, 158

K

K-weight budgets for page downloads, 78
Kafka, processing data streamed to in pipeline architecture, 146
katas, 68
(see also architecture katas)
keys for diagrams, 321
Knuth, Donald, 144
Kops, Micha, 285
kubernan (to steer), 82
kubernetes, 268

L

labels in diagrams, 320
last participant support (LPS), 202
latency
fallacy of zero latency in distributed computing, 125
varying replication latency in space-based architecture, 225
Latency Monkey, 88
layered architecture, 87, 133-141
adding layers, 136
architecture characteristics ratings, 139
ArchUnit fitness function to govern layers, 87
layers of isolation, 135
NetArchTest for layer dependencies, 87
other considerations, 138
self-assessment questions, 376
technical partitioning in Silicon Sandwiches case study, 107
technical partitioning of components, 107
topology, 133-135
use cases for, 139
layered monoliths, 102, 119
layered stack
defined, 343
providing guidance for, 344
layers, drawing tools supporting, 317
leadership skills (see negotiation and leadership skills; software architects)
learnability, 60
usability and, 61
learning something new, 366
least worst architecture, 63, 75
legal requirements, 59
Leroy, Jonny, 104
leverageability, 59
Lewis, James, 245
libraries, 99
shared library for entity objects in service-based architecture, 170
shared library plug-in implementation in microkernel architecture, 154
third-party, in layered stack, 343
lines in diagrams, 319
localization, 59
loggability in Going, Going, Gone case study, 96
logging
distributed, 131
statements generated with Hazelcast in space-based architecture, 217
logical partitioning, database in service-based architecture, 170

M

magnets in drawing tools, 318
maintainability, 59
defined, 62
making teams effective (see development teams, making effective)
managing architecture decision records with ADR-tools blog post, 285
maturity, 61
McAfee, Andrew, 371
McCabe, Thomas, Sr., 79
McCullough, Matthew, 321
McIlroy, Doug, 144
mean-time-to-recovery (MTTR)
high MTTR in layered architecture, 141
high MTTR in pipeline architecture, 148
mediator topology (event-driven architecture), 180, 185-195
trade-offs, 195
meetings, controlling, 361-363
member list of processing units, 217
mergers and acquisitions, translation to architecture characteristics, 67
message queues, 122
messages, use for asynchronous communication in microservices, 255
messaging
data pumps implemented as, 220
for item auction system, 31
message flow in orchestration-driven service-oriented architecture, 238

messaging grid, 214
Metaobject protocol, 38
microfrontends, 253
microkernel architecture, 149-161
  architecture characteristics ratings, 160
  contracts between plug-ins and core system, 158
  core system, 150-153
  examples and use cases, 158
  plug-in components, 153-157
  registry, 157
  self-assessment questions, 377
  in Silicon Sandwiches monolithic architectures case study, 272
  topology, 149
microservices, 3
  bounded context in, 93
  components and, 100
  domain partitioning in Silicon Sandwiches case study, 107
  implementation of Going, Going, Gone using, 275
  liaison between operations and architecture, 18
  service-based architecture style and, 163
"Microservices" blog entry, 245
microservices architecture, 245-265
  API layer, 249
  architecture characteristics ratings, 263-265
  bounded context in, 247
    granularity for services, 248
  communication in, 254-263
    choreography and orchestration, 256-259
    transactions and sagas, 260-263
  distributed architecture, 247
  frontends, 253-254
  history of, 245
  operational reuse in, 250
  references on, 265
  self-assessment questions, 379
  topology, 246
mitigation of architecture risk, 307
  availability risk in nurse diagnostics system example, 311
  elasticity of nurse diagnostics system example, 312
  security risks in nurse diagnostics system example, 314
modifiability, 62
modular monoliths, 102
  domain partitioning in Silicon Sandwiches case study, 107
  using in Silicon Sandwich case study, 271
modular programming languages, 38
modularity, 37-53
  about, 38-40
  fitness functions testing, 84-89
    cyclic dependencies example, 84-86
    distance from main sequence example, 86-88
  important, but not urgent in software projects, 82
  maintainability and, 62
  measuring, 40-53
  in pipeline architecture, 147
  rating in microkernel architecture, 161
  self-assessment questions, 374
  in service-based architecture, 177
MongoDB
  database for logging in risk storming example, 302, 305
  persisting data to in pipeline architecture, 146
monolithic architecture
  C4 diagramming for, 319
  distributed architecture versus, 123-132, 270
    in Going, Going, Gone case study, 115-116
  Silicon Sandwiches case study, 271-274
"More Shell, Less Egg" blog post, 144
most frequently used (MFU) cache, 230
most recently used (MRU) cache, 230
Mule ESB, 186
Myers, Glenford J., 37
The Mythical Man Month (Brooks), 335

N

n-tiered architecture (see layered architecture)
Naked Objects framework, 111
name conflicts in programming platforms, 39
namespaces, 39
  separate, for plug-in components of microkernel architecture, 154
near-cache, considerations in space-based architecture, 230
negotiation and leadership skills, 347-363
  negotiation and facilitation, 347-353

negotiating with business stakeholders, 348
negotiating with developers, 351
negotiating with other architects, 350
self-assessment questions, 381
software architect as leader, 353-360
being pragmatic, yet visionary, 355
C's of architecture, 353
leading teams by example, 357-360
.NET
Naked Objects framework, 111
NetArchTest tool, 87
Netflix, Chaos Monkey and Simian Army, 88
networks
fallacies in distributed computing
the network is homogeneous, 131
the network is reliable, 124
the network is secure, 127
the topology never changes, 128
there is only one administrator, 129
transport cost is zero, 130
network-level protocols, three-tier architecture and, 122
Neward, Ted, 68, 372
nonfunctional requirements, architecturally significant decisions impacting, 284
nonrepudiation, 61
Nygard, Michael, 284

O

OmniGraffle, 317
on-premises implementations of space-based architecture, 226
online auction system example (space-based architecture), 232
open versus closed layers, 136
operating systems, 247
before open source, expensive licensing of, 235
in technology platforms, 368
operational measures of architecture characteristics, 78
operations
intersection with software architecture, 17
operational architecture characteristics, 58
software architecture and, 13
Oracle BPEL Process Manager, 187
Oracle Coherence, 213
orchestration and choreography
error handling and orchestration in event mediators, 186
in microservices' communication, 256
orchestration in space-based architecture, 234
services in service-based architecture, 177
orchestration engines, 238
acting as giant coupling points, 242
orchestration-driven service-oriented architecture, 235-243
architecture characteristics ratings, 241
history and philosophy of, 235
reuse and coupling in, 239-241
self-assessment questions, 379
taxonomy, 236-239
application services, 237
infrastructure services, 237
message flow, 238
orchestration engine, 238
topology, 236
over-specifying architecture characteristics, 73
overall costs, 140
(see also cost)
in microkernel architecture, 160
in pipeline architecture, 147
in service-based architecture, 176

P

packages, 38
plug-in components of microkernel architecture implemented as, 154
Page-Jones, Meilir, 92
partitioning of components
in microkernel architecture, 161
in microservices architecture, 249
in orchestration-driven service-oriented architecture, 242
in space-based architecture, 234
Silicon Sandwiches case study, domain and technical partitioning, 271
people skills, 357-360
performance
as an architecture characteristic, 58, 73
domain concerns translated to architecture characteristics, 67
in Going, Going, Gone case study, 96
multiple, nuanced definitions of, 78
operational measures of, 78
performance efficiency, defined, 61

rating in event-driven architecture, 209
rating in layered architecture, 141
rating in microkernel architecture, 161
rating in microservices arhitecture, 264
rating in orchestration-driven service-oriented architecture, 243
rating in service-based architecture, 233
trade-offs with security, 63
persistence
in component-based thinking, 103
persistence layer, 104
person's network of contact between people, 371
physical topology variants in layered architecture, 134
pipeline architecture, 143-148
architecture characteristics ratings, 146
example, 145-146
self-assessment questions, 376
topology, 143
pipes and filters architecture (see pipeline architecture)
pipes in pipeline architecture, 143
plug-in components, microkernel architecture, 153-157
in Silicon Sandwiches case study, 273
plus (+) and minus (–) sign indicating risk direction, 300
PMD, 158
point-to-point plug-in components in microkernel architecture, 154
politics, understanding and navigating, 12
portability
as structural architecture characteristic, 59
defined, 62
pragmatic, being, 355
presentation layer in core system of microkernel architecture, 152
Presentation Patterns (Ford et al.), 321
presentations versus infodecks, 324
presenting architecture, 315
(see also diagramming and presenting architecture)
privacy, 59
process loss, 335
process measures for architectural characteristics, 81
processing event, 181, 186
processing grid, 218
processing units, 213
containing same named cache, data collisions and, 226
data readers and, 223
data replication within, 217
data writers and, 221
flexibility of, 234
loss of, 218
producer filters, 144
proof-of-concepts (POCs), 35
protocol-aware heterogeneous interoperability, 254
Pryce, Nat, 285
pseudosynchronous communications, 204
publish/subscribe messaging model in broker toplogy of event-driven architecture, 181

Q

quantum, 92
(see also architecture quantum)
queues and topics, trade-offs between, 33

R

radar for personal use, developing, 367-371
open source visualization bits, 371
ThoughtWorks Technology Radar, 368
random replacement eviction policy in front cache, 230
React framework, 254
recoverability, 58
Italy-ility and, 60
performance and, 67
reliability and, 61
registry for plug-ins in microkernel architecture, 157
reliability, 57, 58
availability versus, 60
defined, 61
in Going, Going, Gone case study, 97
in Going, Going, Gone: discovering commponents case study, 114
implicit architecture characteristic, 73
in service-based architecture, 176
in layered architecture, 141
performance and, 67
rating in microkernel architecture, 161
rating in microservices architecture, 263
rating in pipeline architecture, 148
remote access

plug-ins in microkernel architecture, 155
services in service-based architecture, 164
replaceability, 62
replication
replicated vs. distributed caching in space-based architecture, 227-230
varying latency in space-based architecture, 225
replication unit in processing unit, 213
representational consistency, 315
request orchestrator, 179
request processors, 179
request-based model (applications), 179
choosing between event-based and, 206
scaling to meet increased loads in web applications, 211
request-reply messaging in event-driven architecture, 204
requirements
assigning to components, 109
extracting architecture concerns from, 67-69
resilience, 60
resource utilization, 61
responsibility, diffusion of, 337
REST
access to services in service-based architecture via, 164
remote plug-in access via, 155
restructuring of components, 109
reusability, 62
reuse and coupling, 246
operational reuse in microservices architecture, 250
in orchestration-driven service-oriented architecture, 239-241
Richards, Mark, 74
risk (architecture), analyzing (see architecture risk, analyzing)
risk assessments (for architecture risk), 298
risk matrix (for architecture risk), 297
risk storming, 302-308
consensus activity, 304
examples, 308-314
availability of nurse diagnostics system, 310
elasticity of nurse diagnostics system, 312
nurse diagnostics system, 308
identifying areas of risk, 303
mitigation of risk, 307
primary activities in, 303
robustness, 58
roles and responsibilities, analyzing for components, 109
Roosevelt, Theodore, 363
Ruby on Rails, mappings from website to database, 111

S

sagas (transactional), 132
saga pattern in microservices, 261
scalability, 58
elasticity versus, 70
in Going, Going, Gone case study, 96
limits for web-based topologies, 211
low rating in layered architecture, 141
low rating in microkenel architecture, 160
low rating in pipeline architecture, 148
performance and, 67
rating in event-driven architecture, 209
rating in microservices arhitecture, 264
rating in orchestration-driven service-oriented architecture, 243
rating in service-based architecture, 176
rating in space-based architecture, 233
solving issues with space-based architecture, 212
scale, 13
elastic, 14
Schutta, Nathaniel, 321
Second Law of Software Architecture, 19
security, 57
consideration as architecture characteristic, 73
in cross-cutting architectural characteristics, 59
defined, 61
in Going, Going, Gone case study, 96
risks in nurse diagnostics system example, 313
trade-offs with performance, 63
Security Monkey, 88
self-assessment questions, 373-381
separation of concerns, 135
separation of technical concerns, 104
serialization in Java, 123
service discovery, 252

Service Info Capture filter, 146
service locator pattern, 164
service meshes, 251
service plane in microservices, 251
service-based architecture, 163-177
architecture characteristics ratings, 174
example, 172-173
self-assessment questions, 377
service design and granularity, 167-169
topology, 163
topology variants, 165-167
use cases, 177
services, 100
adding new services layer to architecture, 137
decoupling in microservices architecture, 247
granularity for, in microservices, 248
in microkernel architecture core system, 151
in microservices implementation of Going, Going, Gone, 276
risk assessment based on, 299
in service-based architecture style, 163
service instances using named cache in space-based architecture, 217
data collisions and, 225
service layer, 104
shapes in diagrams, 320
shells, use with pipeline architecture, 144
sidecar pattern, 250
Silicon Sandwiches case study, 69-74
implicit architecture characteristics, 73
monolithic architectures, 271-274
Silicon Sandwiches partitioning case study, 105-107
domain partitioning, 107
Simian Army, 88
origin of, 88
simplicity
in event-driven architecture, 209
in layered architecture, 140
in microkernel architecture, 160
in orchestration-driven service-oriented architecture, 243
in pipeline architecture, 147
in service-based architecture, 176
in space-based architecture, 234
slides in presentations, 324
social media, using, 371
software architects, 1
boundary types created for development teams, 325
developing a career path, 365-372
developing a personal radar, 367-371
parting advice, 372
twenty-minute rule, 365-367
using social media, 371
expectations of, 8-13
integration with development team, 360-363
leadership skills, 353-360
being pragmatic, yet visionary, 355
C's of architecture, 353
leading teams by example, 357-360
level of control on development teams, 331-335
leveraging checklists for development teams, 338-343
making development teams effective, summary of important points, 346
negotiation and facilitation skills, 347-353
negotiating with business stakeholders, 348
negotiating with developers, 351
negotiating with other architects, 350
observing development team warning signs, 335-338
personality types, 326-331
armchair architects, 328
control freak, 327
effective architects, 330
providing guidance to development teams, 343-346
role in components, 101-107
software architecture
about, 3-7
dynamic nature of, 2
emerging standards for diagramming, 293
intersection with other departments, 13
data, 19
engineering practices, 14
operations/DevOps, 17
software development process, 18
introduction to, self-assessment questions, 373
lack of clear definitions in, 63
laws of, 19
software development

changes in the ecosystem, 267
development process and software architecture, 18
software release checklist, 342
space-based architecture, 211-234
advantages of, 212
architecture characteristics ratings, 233
cloud vs. on-premises implementations, 226
data collisions, 224-226
general topology, 212
data pumps, 219
data readers, 222
data writers, 221
processing unit, 213
virtualized middleware, 214-219
implementation examples, 231-232
concert ticketing system, 231
online auction system, 232
near-cache, considerations with, 230
replicated vs. distributed caching in, 227-230
self-assessment questions, 378
Spring Integration, 186
stamp coupling, 126
standards, using ADRS for, 293
static connascence, 92
status of an ADR, 286
stencils/templates, drawing tools supporting, 317
strangler pattern, 18
strategic positioning, justification for architecture decisions, 283
strategy pattern, 12
structural architecture characteristics, 59
structural measures of architectural characteristics, 79, 81
structure of the system, 5
structure, architecturally significant decisions impacting, 284
structured programming languages, 38
Sun Tzu, 350
supportability, 59, 60
Swedish warship (Vasa) case study, 66
synchronous comunication, 254, 270
in microservices implementation of Going, Going, Gone, 276
synchronous send in event-driven architecture, 201
synchronous connascence, 92, 93
system level, architecture characteristics at, 91

T

teams, making effective (see development teams, making effective)
technical and business justifications for architecture decisions, 282, 343
technical breadth, 25-30
technical debt, tackling, 35
technical decisions versus architecture decisions, 284
technical knowledge, solving technical issues and, 357
technical partitioning (components)
domain partitioning versus, 249
in event-driven architecture, 208
in microkernel architecture, 161
in orchestration-driven service-oriented architecture, 241
in pipeline architecture, 146
in layered architecture, 135
in Silicon Sandwiches monolithic architectures case study, 271
in Silicon Sandwiches patitioning case study, 107
technical top-level partitioning, 103
technology bubbles, 367
technology changes, impact on architecture styles, 268
Technology Radar (see ThoughtWorks Technology Radar)
Template Method design pattern, 74
temporary queues, 205
test-driven development (TDD), resulting in less complex code, 81
testability
low rating in layered architecture, 140
process measures of, 81
rating in event-driven architecture, 209
rating in microkernel architecture, 161, 263
rating in orchestration-driven service-oriented architecture, 242
rating in pipeline architecture, 147
rating in service-based architecture, 176
rating in space-based architecture, 234
tester filters, 144
testing
rating in space-based architecture, 234
unit and functional testing checklist, 341

ThoughtWorks Build Your Own Radar tool, 371
ThoughtWorks Technology Radar, 366, 367-371
three-tier architecture, 122
  language design and long-term implications, 123
time and budget, translation to architecture characteristics, 67
time behavior, 61
time to market
  agility versus, 67
  justification for architecture decisions, 283
  translation to architecture characteristics, 67
timeouts, 124
titles in diagrams, 319
top-level partitioning in an architecture, 102
  domain partitioning, 104
topics and queues, trade-offs between, 33
topology (network), changes in, 128
trade-offs in software architecture, 19, 372
  analyzing, 30
  architecture characteristics and least worst architecture, 63
  design versus, 74
  reuse and coupling, 246
transactional sagas, 132
transactions
  ACID or BASE, 168
  across boundaries in microservices, not recommended, 247
  and bounded contexts in microservices, 248
  difficulty of distributed transactions, 243
  distributed, 132
  in orchestration-driven service-oriented architecture, 238
  in service-based architecture, 177
  lack of ability to restart in broker toplogy of event-driven architecture, 185
  and sagas in microservices' communication, 260-263
transfomer filters, 144
transitions from presentation tools, 321
transport cost in distributed computing, 130
traveling salesperson problem, 83
tuple space, 212
twenty-minute rule, 365-367
two-tier architecture, 121

U

ubiquitous language, use of, 63
Unified Modeling Language (UML), 318
unitary architecture, 121
unknown unknowns in software systems, 15
unplanned downtime, 349
upgradeability, 59
Uptime Calculator transformer filter, 146
Uptime filter (tester filter), 146
usability
  architecture characteristics and, 73
  defined, 61
usability/achievability, 60
user error protection, 61
user interfaces (UIs)
  access to services in service-based architecture via, 164
  as part of bounded context in DDD, 253
  in distributed architecture Going, Going, Gone case study, 275
  microservices with monolithic UI, 253
  separate UI in microkernel architecture, 152
  variants in service-based architecture, 165
user satisfaction
  justification for architecture decisions, 283
  translation to arcitecture characteristics, 67

V

value-driven messages, 220
variance, 7
Vasa case study, 66
virtualized middleware, 213, 214-219
  data grid, 215
  deployment manager, 219
  messaging grid, 214
  processing grid, 218
visionary, being, 355

W

web applications, scaling to meet increased loads, 211
web browsers, using microkernel architecture, 158
web servers
  and browser architecture, 122
  scaling, problems with, 211
Weinberg, Gerald, 357

What Every Programmer Should Know About Object Oriented Design (Page-Jones), 92
Why is more important than How, 19
Wildfly application server, 161
workflows
database relationships incorrectly identified as, 111
in technical and domain-partitioned architecture, 104
workflow approach to designing components, 112
workflow event pattern, 197-200
workflow delegate, 197
workflow processor, 198

Y

Yoder, Joseph, 120

About the Authors

Mark Richards is an experienced hands-on software architect involved in the architecture, design, and implementation of microservices and other distributed architectures. He is the founder of *DeveloperToArchitect.com*, a website devoted to assisting developers in the journey from developer to a software architect.

Neal Ford is director, software architect, and meme wrangler at ThoughtWorks, a global IT consultancy with an exclusive focus on end-to-end software development and delivery. Before joining ThoughtWorks, Neal was the chief technology officer at The DSW Group, Ltd., a nationally recognized training and development firm.

Colophon

The animal on the cover of *Fundamentals of Software Engineering* is the red-fan parrot (*Deroptyus accipitrinus*), a native to South America where it is known by several names such as *loro cacique* in Spanish, or *anacã*, *papagaio-de-coleira*, and *vanaquiá* in Portugese. This New World bird makes its home up in the canopies and tree holes of the Amazon rainforest, where it feeds on the fruits of the *Cecropia* tree, aptly known as "snake fingers," as well as the hard fruits of various palm trees.

As the only member of the genus *Deroptyus*, the red-fan parrot is distinguished by the deep red feathers that cover its nape. Its name comes from the fact that those feathers will "fan" out when it feels excited or threatened and reveal the brilliant blue that highlights each tip. The head is topped by a white crown and yellow eyes, with brown cheeks that are streaked in white. The parrot's breast and belly are covered in the same red feathers dipped in blue, in contrast with the layered bright green feathers on its back.

Between December and January, the red-fan parrot will find its lifelong mate and then begin laying 2-4 eggs a year. During the 28 days in which the female is incubating the eggs, the male will provide her with care and support. After about 10 weeks, the young are ready to start fledging in the wild and begin their 40-year life span in the world's largest tropical rainforest.

While the red-fan parrot's current conservation status is designated as of Least Concern, many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black and white engraving from *Lydekker's Royal Natural History*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

O'REILLY®

There's much more where this came from.

Experience books, videos, live online training courses, and more from O'Reilly and our 200+ partners—all in one place.

Learn more at oreilly.com/online-learning

©2019 O'Reilly Media, Inc. O'Reilly is a registered trademark of O'Reilly Media, Inc. | 175